

Implementation-of-Logistic-Regression-Using-Gradient-Descent

AIM:

To write a program to implement the the Logistic Regression Using Gradient Descent.

Equipments Required:

1. Hardware – PCs
2. Anaconda – Python 3.7 Installation / Jupyter notebook

Algorithm

- 1.Import the data file and import numpy, matplotlib and scipy.
- 2.Visulaize the data and define the sigmoid function, cost function and gradient descent.
- 3.Plot the decision boundary .
- 4.Calculate the y-prediction.

Program:

```
/* Program to implement the the Logistic Regression Using Gradient Descent.
```

```
Developed by: Jagan J P
```

```
RegisterNumber: 212224230099
```

```
*/
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
data = pd.read_csv("Placement_Data.csv")
data.head()
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
categorical_cols = [
```

```

    'gender', 'ssc_b', 'hsc_b', 'hsc_s',
    'degree_t', 'workex', 'specialisation', 'status'
]

for col in categorical_cols:
    data[col] = le.fit_transform(data[col])

data.head()
X = data.drop(['status', 'salary'], axis=1).values
y = data['status'].values.reshape(-1, 1)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X = scaler.fit_transform(X)
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
m, n = X_train.shape    # samples, features
w = np.zeros((n, 1))    # weights
b = 0                   # bias

alpha = 0.01            # learning rate
iterations = 3000        # number of iterations
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
losses = []

for i in range(iterations):
    z = np.dot(X_train, w) + b
    y_hat = sigmoid(z)

    dw = (1/m) * np.dot(X_train.T, (y_hat - y_train))
    db = (1/m) * np.sum(y_hat - y_train)

    w -= alpha * dw
    b -= alpha * db

    loss = -(1/m) * np.sum(
        y_train * np.log(y_hat + 1e-9) +
        (1 - y_train) * np.log(1 - y_hat + 1e-9)
    )
    losses.append(loss)

print("Training completed")
plt.plot(losses)
plt.xlabel("Iterations")
plt.ylabel("Loss")

```

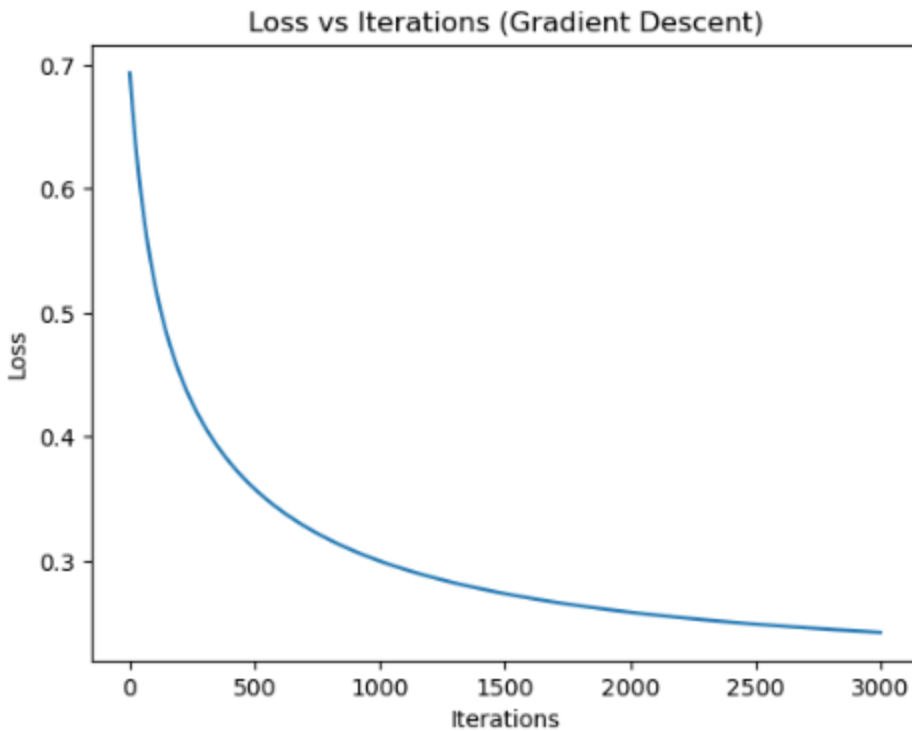
```
plt.title("Loss vs Iterations (Gradient Descent)")
plt.show()
def predict(X):
    z = np.dot(X, w) + b
    y_pred = sigmoid(z)
    return (y_pred >= 0.5).astype(int)
y_pred = predict(X_test)

accuracy = np.mean(y_pred == y_test)
print("Accuracy:", accuracy)
from sklearn.metrics import confusion_matrix, classification_report

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Output:

Training completed



Accuracy: 0.8604651162790697

Confusion Matrix:

```
[[ 9  3]
 [ 3 28]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.75	0.75	0.75	12
1	0.90	0.90	0.90	31
accuracy			0.86	43
macro avg	0.83	0.83	0.83	43
weighted avg	0.86	0.86	0.86	43

Result:

Thus the program to implement the the Logistic Regression Using Gradient Descent is written and verified using python programming.